

Задание №5.

Верификация MVVM-паттерна

Цель задания:

- Ознакомиться со стандартным пакетом [Community MVVM Toolkit](#).
- Упростить реализацию за счет использования пакета *Community MVVM Toolkit*.
- Проверить правильность реализации MVVM-паттерна.

Предыдущие задания предполагали, что реализацию интерфейсов `INotifyPropertyChanged`, `IDataErrorInfo`, а также классов `ICommand` вы пишете самостоятельно. При этом, в качестве рекомендации предлагалось использовать сторонние библиотеки с готовыми классами для реализации MVVM-паттерна, такие как *Community MVVM Toolkit* или *Galasoft.MVVMLight*.

Самостоятельная реализация стандартных интерфейсов полезна для понимания того, как работает MVVM-приложение и какие обязанности лежат на тех или иных сущностях в коде. Однако теперь, когда функциональность нашей просто информационной системы готова, мы можем заменить часть своего кода на использование стандартных фреймворков.

Если при создании решения вы использовали проекты типа NET 6.0 и выше, при выполнении следующих заданий используйте библиотеку *Community MVVM Toolkit*. Если вы использовали NET Framework, вы можете использовать либо *Community MVVM Toolkit*, либо *Galasoft.MVVMLight* – однако в таком случае не все функции *Community MVVM Toolkit* (в частности, специализированные атрибуты) будут доступны в вашем проекте. Об этом будет отдельно указано в пунктах задания. В любом случае, отсутствие данных функций не ухудшит ваше понимание MVVM-паттерна, а лишь не позволит использовать несколько косметических конструкций для упрощения вашего кода. Следующие задания написаны для библиотеки *Community MVVM Toolkit*. При использовании *Galasoft.MVVMToolkit* вы можете легко найти их аналоги по документации к библиотеке.

Задание

1) Подключить через Nuget библиотеку [Community MVVM Toolkit](#).

2) Отнаследовать все VM от [ObservableObject](#) - базовый класс, реализующий интерфейс INotifyPropertyChanged. Так как реализация любой VM предполагает объявление события PropertyChanged и ряда методов, данную реализацию можно вынести в базовый класс. Чтобы самостоятельно не создавать базовый класс, фреймворк Community MVVM Toolkit уже содержит такую реализацию, которая называется ObservableObject.

3) Все коллекции в VM изменить на использование [ObservableCollection<>](#) [\[1, 2\]](#). Класс ObservableCollection является коллекцией, реализующей INotifyPropertyChanged. Таким образом, у неё есть событие PropertyChanged, через которое она уведомляет пользовательский интерфейс об изменениях в коллекции: добавлении, редактировании или удалении объектов в коллекции. Важно: для корректного уведомления о редактировании объектов внутри коллекции необходимо, чтобы внутри коллекции лежали объекты, которые также реализуют интерфейс INotifyPropertyChanged (или наследуются от ObservableObject).

4) Все команды реализовать на основе класса [RelayCommand](#) из Community MVVM Toolkit. Свою реализацию данного класса удалить. В одном из первых заданий предлагалось создавать отдельные классы, реализующие интерфейс ICommand. Это не самый удобный способ реализации команд для пользовательского интерфейса. Наверняка, при изучении источников, указанных в литературе, вы встречались с упоминанием класса RelayCommand – базового класса для создания команд. Его использование значительно упрощает реализацию VM-классов. Если вы не создали данных класс у себя в проекте по примерам из литературы, тогда вам необходимо заменить собственные классы ICommand на использование класса RelayCommand из проекта Community MVVM Toolkit. Если вы реализовывали команды в своей программе с помощью собственной реализации класса RelayCommand, тогда замените свою реализацию на реализацию из стороннего пакета, а свою реализацию удалите.

5) Если используете NET 6.0 и выше, то перепишите свойства VM на использование атрибута [ObservableProperty](#). При необходимости используйте атрибуты NotifyPropertyChangedFor, NotifyCanExecuteChangedFor. Для команд вы также можете воспользоваться атрибутом [RelayCommand](#) (не путать с классом RelayCommand). Если вы используете NET Framework, данные атрибуты не будут работать, поэтому отладьте реализацию своих VM без использования данных атрибутов.

Указанные атрибуты являются «синтаксическим сахаром», которые позволяют сократить реализацию свойств VM до объявления обычных полей. В новых версиях NET это позволяет сократить код VM еще на несколько десятков строк кода.

6) Разделите ваш проект View на три отдельных проекта: Model, ViewModel, View (альт. вариант именования проектов: Contacts.View, Contacts.ViewModel, Contacts.Model). Соответственно:

- Все классы, которые в текущем проекте View относятся к бизнес-логике, должны быть перенесены в проект Model.

- Все классы, которые в текущем проекте View относятся к уровню представления вида, должны быть перенесены в проект ViewModel.

- В проекте View должны остаться только те классы, которые относятся к пользовательскому интерфейсу - окна, элементы управления, конвертеры IValueConverter, сервисные классы, используемые в пользовательском интерфейсе.

- Проекты Model и ViewModel должны быть проектами типа "Class Library" ("Библиотека классов").

- Проекты ViewModel и View должны подключить пакет MVVM Toolkit. Если классы уровня Model реализуют интерфейс INotifyPropertyChanged, IDataErrorInfo или иные интерфейсы MVVM, то данный проект также может подключить пакет MVVM Toolkit.

7) Убедиться, что проект View подключает ViewModel, а ViewModel подключает Model. View также может подключать проект Model при необходимости, но ViewModel и Model не должны подключать проект View - это нарушение паттерна MVVM.

8) Протестировать и отладить программу. Если после разделения программы на отдельные проекты, ваша программа работает корректно с соблюдением всех вышеперечисленных требований — значит, вы правильно реализовали архитектуру решения, и паттерн MVVM полностью выполнен.

Важно отметить, что для небольших приложений разделение программы на три отдельных проекта, как правило, избыточно: все классы могут находиться в одном проекте, или приложение может быть разделено на два проекта Model и View + VM. Однако, с точки зрения данного курса было важно проверить правильность архитектуры и получить навык разделения решения на отдельные модули.

Следующими шагами по изучению паттерна MVVM является создание много-оконного приложения с использованием Dependency Injection и IoC-контейнеров [1, 2, 3]. Данные приёмы позволяют горизонтально масштабировать MVVM-приложение. Изучение данных техник не входит в рамки данного курса заданий, и читателю предлагается изучить их самостоятельно, например, детальное изложение с примерами кода содержится в [Vice].

Литература

1. **Мак-Дональд М.**
WPF 4: Windows Presentation Foundation в .NET 4.0 с примерами на C# 2010 для профессионалов пер. с англ. – М.: ООО «И.Д. Вильямс», 2011. – 1024 с. : ил. – парал. тит. англ.
2. **Metanit.com. Руководство по WPF**
/ Metanit.com. – URL: <https://metanit.com/sharp/wpf/>
3. **Vice R., Siddiqi M.S.**
MVVM Survival Guide for Enterprise Architectures in Silverlight and WPF. – Birmingham-Mumbai: Packt Publishing Ltd., 2012. – 491 p.
4. **Microsoft. Introduction to the MVVM Toolkit**
/ learn.microsoft.com. – URL: <https://learn.microsoft.com/en-us/dotnet/communitytoolkit/mvvm/>